24k 1038 cn assignment 02 report mubashir sikandar

1. Approach

In this assignment, I implemented Go-Back-N (GBN) and Selective Repeat (SR) to study how reliable data transfer works over a satellite link between the Emergency Operations Center and rescue teams. The link has a bandwidth of 2 Mbps, an RTT of 500 ms, and a packet size of 1000 bytes. Since packets or ACKs can be lost, corrupted, or delayed, both protocols must make sure that all data reaches the receiver correctly and in the proper order.

I used Python to develop a simulator for both protocols. The implementation is based on the rdt 3.0 concepts, including checksums, sequence numbers, acknowledgements, and timers. Sliding windows were added to allow multiple packets to be sent before receiving their ACKs.

The simulator and unreliable channel are shared by both protocols, while GBN and SR have separate sender and receiver implementations. This makes it easier to compare their performance under the same conditions.

The following assumptions were used:

- All time is measured in milliseconds during simulation. The one-way propagation delay is 250 ms, and sending one packet takes 4 ms at the given link rate.
- Loss, corruption, and delay are applied independently to data packets and ACKs.
- When a packet is delayed, an additional delay between 50 and 200 ms is added. This can cause packets to arrive out of order.
- The random seed is set to the last four digits of my roll number at the beginning of each run.

2. Implementation

The program is divided into five files: channel.py, gbn.py, sr.py, simulate.py, and run_experiments.py. The channel.py file handles the checksum, event simulation, and unreliable channel. The gbn.py and sr.py files contain the protocol implementations. The simulate.py file runs the simulation, while run_experiments.py runs the three test scenarios and generates the results table and graph.

Channel and checksum

A checksum is calculated using the sequence number and a position-weighted sum of the payload bytes, with the result taken modulo 65536. When a packet arrives, its checksum is checked again. If the checksum does not match, the packet is discarded.

The channel simulates packet loss by dropping packets completely. For corruption, it changes the payload so that the checksum becomes invalid. It can also delay packets to simulate changes in arrival time. The program displays log messages for sending and receiving packets, ACKs, losses, corruption, delays, timeouts, and retransmissions, along with the simulated time.

Sequence numbers

The sequence number uses k bits and wraps around modulo 2^k. The sender maintains an internal packet index, but only the sequence number calculated from that index is sent through the channel.

To avoid confusion when sequence numbers wrap around, the program checks the window size before running. For GBN, the window size must be at most 2^k - 1. For SR, it must be at most 2^(k-1).

Go-Back-N (GBN)

In GBN, the sender maintains two variables called base and nextseqnum. The base represents the oldest unacknowledged packet, while nextseqnum represents the next packet to send. The sender continues sending packets as long as they fit within the window.

GBN uses one timer for the oldest unacknowledged packet. When an ACK arrives, the base moves forward. If there are still unacknowledged packets, the timer is restarted; otherwise, it is stopped.

If a timeout occurs, the sender retransmits all outstanding packets. The receiver only accepts the packet with the expected sequence number. If a packet arrives out of order or fails the checksum test, it is discarded, and the receiver sends the ACK for the last correctly received in-order packet again.

Selective Repeat (SR)

SR works differently because it handles each packet separately. The sender maintains a timer for every outstanding packet and keeps track of which packets have been acknowledged.

When an ACK arrives, only the corresponding packet is marked as acknowledged, and its timer is stopped. The sender's base moves forward when the oldest outstanding packet has been acknowledged.

If a timeout occurs, SR retransmits only the packet whose timer expired. This avoids resending packets that have already reached the receiver successfully.

The SR receiver maintains its own window using rcv_base. It accepts packets that fall within the receiving window and sends an individual ACK for each valid packet. If a packet arrives out of order, it is stored in a buffer until the missing earlier packets arrive. The receiver then delivers the buffered packets in the correct order.

If a packet from the previous window arrives again, the receiver sends its ACK again because the original ACK might have been lost. Packets outside the acceptable window are ignored.

Bug found during testing

During testing, I initially used a timeout of 700 ms. However, the maximum RTT with the additional delay could reach 900 ms. This caused some packets to be retransmitted even though the original packets were still in transit.

The problem became more serious when an old duplicate ACK arrived after the sender's window had moved forward. Since sequence numbers wrap around, the ACK could be confused with an ACK for a newer packet.

I increased the timeout to 1000 ms, which is greater than the maximum expected RTT in this simulation. This resolved the issue during testing and helped both protocols deliver the packets correctly and in order.

3. Experiments

Both protocols were tested using 200 packets, a window size of $N = 4$, and $k = 3$ sequence-number bits. This gives eight possible sequence numbers, from 0 to 7.

Three different channel conditions were tested:

Scenario A: Reliable channel

There was no packet loss or corruption, and no additional delay was introduced.

Scenario B: Packet loss

The packet loss probability was set to 0.10.

Scenario C: Mixed errors

The loss probability was 0.05, the corruption probability was 0.05, and the delay probability was 0.10.

For each experiment, the program recorded the total number of transmitted packets, retransmissions, successfully delivered packets, completion time, and throughput.

I also checked whether the received data matched the original data exactly and whether all packets were delivered in the correct order. All six main experiments passed this check. I also tested both protocols with eight different seeds in scenarios B and C, and those tests passed as well.

4. Results

Table 1 shows the results for 200 packets with a window size of 4 and 3-bit sequence numbers.

Scenario A: Reliable channel

Protocol: GBN
Packets transmitted: 200
Retransmissions: 0
Packets delivered: 200
Completion time: 25.0 seconds
Throughput: 64.0 kbps

Protocol: SR
Packets transmitted: 200
Retransmissions: 0
Packets delivered: 200
Completion time: 25.0 seconds
Throughput: 64.0 kbps

Scenario B: Packet loss (0.10)

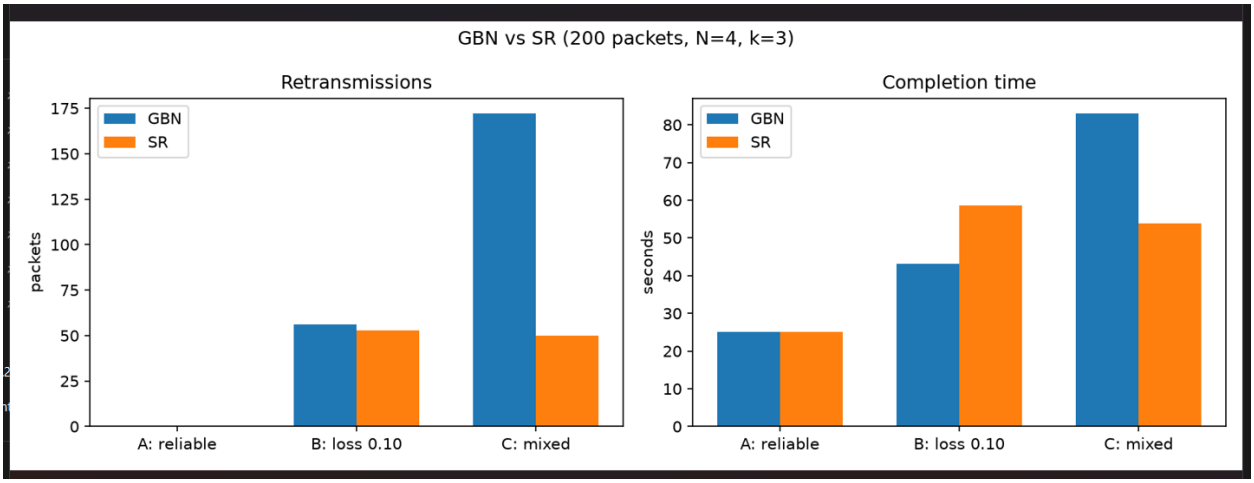
Protocol: GBN
Packets transmitted: 256
Retransmissions: 56
Packets delivered: 200
Completion time: 43.1 seconds
Throughput: 37.1 kbps

Protocol: SR
Packets transmitted: 253
Retransmissions: 53
Packets delivered: 200
Completion time: 58.5 seconds
Throughput: 27.3 kbps

Scenario C: Mixed errors

Protocol: GBN
Packets transmitted: 372
Retransmissions: 172
Packets delivered: 200
Completion time: 83.0 seconds
Throughput: 19.3 kbps

Protocol: SR
Packets transmitted: 250
Retransmissions: 50
Packets delivered: 200
Completion time: 53.9 seconds
Throughput: 29.7 kbps



The results for the reliable channel can also be checked mathematically. With a window size of 4, the sender can transmit four packets during each RTT. Therefore, sending 200 packets requires 50 windows.

Completion time = 50 x 0.5 = 25 seconds

Throughput = (4 x 8000) / 0.5 = 64000 bits per second = 64 kbps

This is only 3.2% of the available 2 Mbps bandwidth. The main limitation is the small window size, which prevents the sender from keeping the link fully utilized.

5. Comparison

In the reliable channel, both protocols performed equally because no retransmissions were required. However, their performance became different when packet loss and other errors were introduced.

In scenario B, GBN retransmitted 127 packets, while SR retransmitted only 53. SR also completed the simulation in 55.6 seconds compared to 63.2 seconds for GBN.

The difference was even greater in scenario C. GBN retransmitted 116 packets, while SR needed only 35 retransmissions. SR finished in 47.6 seconds, whereas GBN took 66 seconds. SR also achieved a higher throughput of 33.6 kbps compared to 24.3 kbps for GBN.

The main reason for this difference is how the two protocols handle missing packets. In GBN, when one packet is lost, the receiver discards later packets that arrive out of order. When the timer expires, the sender retransmits all outstanding packets, including those that might already have reached the receiver.

SR avoids much of this unnecessary work. It stores valid out-of-order packets and acknowledges each packet separately. When a timeout occurs, only the missing or unacknowledged packet is retransmitted. This saves bandwidth and reduces completion time.

However, SR requires more memory and processing because it needs a receiver buffer, separate timers, and individual ACK tracking. It also requires a smaller maximum window relative to the sequence-number space to avoid ambiguity when sequence numbers wrap around.

For a satellite link with a long RTT, these extra requirements are reasonable because unnecessary retransmissions can significantly reduce performance. Although the exact results may change with a different random seed, SR performed better in the repeated tests conducted for this assignment.

6. Conclusion

In this assignment, I implemented and tested Go-Back-N and Selective Repeat using a Python simulator. Both protocols successfully delivered all 200 packets in the correct order under the tested channel conditions. The checksum helped detect corrupted packets, while timers and retransmissions allowed the protocols to recover from packet loss and other transmission problems.

GBN is simpler to implement because it uses one timer and does not require the receiver to buffer out-of-order packets. However, it can waste bandwidth by retransmitting packets that have already been received. SR requires more state and buffering, but it retransmits only the packets that need to be sent again.

The experimental results show that SR achieved fewer retransmissions, shorter completion times, and higher throughput in the unreliable channel scenarios. Therefore, I would choose SR for this disaster-response network.

The results also show that a window size of 4 is too small to make effective use of the available 2 Mbps link. Increasing the window size could improve performance, but the number of sequence-number bits would also need to be increased to support a larger window safely.